

.NET C# Basics and OOP Tutorial for Beginners

Complete Real-Time Developer Guide

1. Introduction to .NET

.NET is a software development platform developed by [Microsoft](#) for building:

- Web Applications
- Desktop Applications
- Mobile Applications
- Cloud Applications
- APIs
- Microservices
- AI Applications
- Enterprise Software

.NET Versions

Version	Description
.NET Framework	Windows-only legacy framework
.NET Core	Cross-platform version
.NET 5+	Unified modern platform
.NET 6	LTS (Long-Term Support)
.NET 8	Latest LTS Version

2. What is C#?

C# (C Sharp) is an Object-Oriented Programming Language used to develop applications on .NET.

Features

- Simple
 - Type Safe
 - Object Oriented
 - Secure
 - Scalable
 - Cross Platform
-

3. First C# Program

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Hello World");
    }
}
```

Output

Hello World

4. Variables

Variables store data.

```
string name = "Parva";
int age = 30;
double salary = 50000.50;
bool isActive = true;
```

```
Console.WriteLine(name);
```

5. Data Types

Value Types

```
int number = 10;  
double amount = 100.50;  
char grade = 'A';  
bool status = true;
```

Reference Types

```
string name = "John";  
object obj = "Hello";
```

6. Operators

Arithmetic Operators

```
int a = 10;  
int b = 5;  
  
Console.WriteLine(a + b);  
Console.WriteLine(a - b);  
Console.WriteLine(a * b);  
Console.WriteLine(a / b);
```

Comparison Operators

```
Console.WriteLine(a > b);  
Console.WriteLine(a < b);  
Console.WriteLine(a == b);
```

Logical Operators

```
bool result = true && false;  
bool result2 = true || false;
```

7. Conditional Statements

If Statement

```
int age = 18;
```

```
if(age >= 18)
{
    Console.WriteLine("Eligible");
}
```

If Else

```
if(age >= 18)
{
    Console.WriteLine("Eligible");
}
else
{
    Console.WriteLine("Not Eligible");
}
```

Switch

```
int day = 2;
```

```
switch(day)
{
    case 1:
        Console.WriteLine("Monday");
        break;

    case 2:
        Console.WriteLine("Tuesday");
        break;

    default:
        Console.WriteLine("Invalid");
        break;
}
```

8. Loops

For Loop

```
for(int i=1;i<=5;i++)
{
    Console.WriteLine(i);
}
```

While Loop

```
int i = 1;

while(i<=5)
{
    Console.WriteLine(i);
    i++;
}
```

Foreach Loop

```
string[] names =
{
    "John",
    "David",
    "Smith"
};

foreach(string name in names)
{
    Console.WriteLine(name);
}
```

9. Methods

```
public static void Greet()
{
    Console.WriteLine("Welcome");
}
```

Call Method

```
Greet();
```

Method with Parameters

```
public static int Add(int a,int b)
{
    return a + b;
}
```

Usage

```
int result = Add(10,20);
```

10. Arrays

```
int[] numbers =
{
    10,
    20,
    30,
    40
};
```

```
foreach(var num in numbers)
{
    Console.WriteLine(num);
}
```

11. Exception Handling

```
try
{
    int a = 10;
    int b = 0;

    int c = a / b;
}
catch(Exception ex)
{
}
```

```
    Console.WriteLine(ex.Message);  
}  
finally  
{  
    Console.WriteLine("Completed");  
}
```

Object Oriented Programming (OOP)

OOP is the foundation of C#.

Four Pillars

1. Encapsulation
 2. Abstraction
 3. Inheritance
 4. Polymorphism
-

12. Class and Object

Class

Blueprint of an object.

```
public class Employee  
{  
    public string Name;  
    public int Age;  
}
```

Object

```
Employee emp = new Employee();
```

```
emp.Name = "Parva";  
emp.Age = 30;
```

13. Constructor

Automatically called when object is created.

```
public class Employee
{
    public Employee()
    {
        Console.WriteLine("Constructor Called");
    }
}
```

Usage

```
Employee emp = new Employee();
```

14. Encapsulation

Wrapping data and methods together.

```
public class Employee
{
    private string name;

    public string Name
    {
        get { return name; }
        set { name = value; }
    }
}
```

Usage

```
Employee emp = new Employee();
```

```
emp.Name = "Parva";
```

Benefits:

- Security
- Data Protection
- Controlled Access

15. Abstraction

Hide implementation details.

```
public abstract class Vehicle
{
    public abstract void Start();
}
```

Derived Class

```
public class Car : Vehicle
{
    public override void Start()
    {
        Console.WriteLine("Car Started");
    }
}
```

Usage

```
Vehicle v = new Car();
v.Start();
```

16. Inheritance

Acquire properties from parent class.

```
public class Person
{
    public void Display()
    {
        Console.WriteLine("Person Class");
    }
}
```

Child Class

```
public class Employee : Person
{
}
```

Usage

```
Employee emp = new Employee();
```

```
emp.Display();
```

17. Polymorphism

One method with many forms.

Method Overloading

```
public class Calculator
{
    public int Add(int a,int b)
    {
        return a + b;
    }

    public int Add(int a,int b,int c)
    {
        return a + b + c;
    }
}
```

Method Overriding

```
public class Animal
{
    public virtual void Sound()
    {
        Console.WriteLine("Animal Sound");
    }
}

public class Dog : Animal
{
    public override void Sound()
```

```
{  
    Console.WriteLine("Bark");  
}
```

Usage

```
Animal animal = new Dog();
```

```
animal.Sound();
```

Output

Bark

18. Interface

100% abstraction contract.

```
public interface ILogger  
{  
    void Log();  
}
```

Implementation

```
public class FileLogger : ILogger  
{  
    public void Log()  
    {  
        Console.WriteLine("Logging");  
    }  
}
```

Usage

```
ILogger logger = new FileLogger();
```

```
logger.Log();
```

19. Collections

List

```
List<string> employees =  
new List<string>();
```

```
employees.Add("John");  
employees.Add("David");
```

Dictionary

```
Dictionary<int,string> employees =  
new Dictionary<int,string>();
```

```
employees.Add(1,"John");  
employees.Add(2,"David");
```

20. Real-Time Employee Management Example

```
public class Employee  
{  
    public int Id { get; set; }  
  
    public string Name { get; set; }  
  
    public decimal Salary { get; set; }  
  
    public void Display()  
    {  
        Console.WriteLine(  
            $"{Id} - {Name} - {Salary}");  
    }  
}
```

Program

```
class Program  
{  
    static void Main()  
    {
```

```
List<Employee> employees =  
    new List<Employee>();  
  
employees.Add(new Employee  
{  
    Id = 1,  
    Name = "John",  
    Salary = 50000  
});  
  
employees.Add(new Employee  
{  
    Id = 2,  
    Name = "David",  
    Salary = 70000  
});  
  
foreach(var emp in employees)  
{  
    emp.Display();  
}  
}
```

